

Var

变量

数据类型

C++ 的类型系统由如下几部分组成：

1. 基础类型（括号内为代表关键词/代表类型）
 1. 无类型/`void` 型 (`void`)
 2. （C++11 起）空指针类型 (`std::nullptr_t`)
 3. 算术类型
 1. 整数类型 (`int`)
 2. 布尔类型/`bool` 型 (`bool`)
 3. 字符类型 (`char`)
 4. 浮点类型 (`float, double`)
2. 复合类型²

布尔类型

一个 `bool` 类型的变量取值只可能为两种：`true` 和 `false`。

一般情况下，一个 `bool` 类型变量占有 1 字节（一般情况下，1 字节 = 8 位）的空间。

▼ Tip

可通过头文件 `<limits>`(C++)/`<limits.h>`(C) 中的宏常量 `CHAR_BIT` 获取字节的位数。

▼ C 语言的布尔类型

另请参阅 [C++ 与其他常用语言的区别 - bool](#)。

C 语言最初是没有布尔类型的，直到 C99 时才引入 `_Bool` 关键词作为布尔类型，其被视作无符号整数类型。

▼ Note

C 语言的 `bool` 类型从 C23 起不再使用整型的零与非零值定义，而是定义为足够储存 `true` 和 `false` 两个常量的类型。

为方便使用，`stdbool.h` 中提供了 `bool, true, false` 三个宏，定义如下：

```
1 #define bool _Bool
2 #define true 1
3 #define false 0
```

这些宏于 C23 中移除，并且 C23 起引入 `true, false` 和 `bool` 作为关键字，同时保留 `_Bool` 作为替代拼写形式¹。

另外，C23 起还可以通过 `<limits.h>` 中的宏常量 `BOOL_WIDTH` 获取布尔类型的位宽。

整数类型

用于存储整数。最基础的整数类型是 `int`。

▼ 注意

由于历史原因，C++ 中布尔类型和字符类型会被视作特殊的整型。

在几乎所有的情况下都 **不应该** 将除 `signed char` 和 `unsigned char` 之外的字符类型作为整型使用。

整数类型一般按位宽有 5 个梯度：`char,short,int,long,long long`。

C++ 标准保证 `1 == sizeof(char) <= sizeof(short) <= sizeof(int) <= sizeof(long) <= sizeof(long long)`

由于历史原因，整数类型的位宽有多种流行模型，为解决这一问题，C99/C++11 引入了 [定宽整数类型](#)。

▼ `int` 类型的大小

在 C++ 标准中，规定 `int` 的位数 **至少** 为 位。

事实上在现在的绝大多数平台，`int` 的位数均为 位。

对于 `int` 关键字，可以使用如下修饰关键字进行修饰：

符号性：

- `signed`：表示带符号整数（默认）；
- `unsigned`：表示无符号整数。

大小：

- `short`：表示 **至少** 位整数；
- `long`：表示 **至少** 位整数；
- （C++11 起）`long long`：表示 **至少** 位整数。

下表给出在 **一般情况下**，各整数类型的位宽和表示范围大小（少数平台上一些类型的表示范围可能与下表不同）：

类型名	等价类型	位宽（C++ 标准）	位宽（常见）	位宽（较罕见）
<code>signed char</code>	<code>signed char</code>	-	-	
<code>unsigned char</code>	<code>unsigned char</code>	-	-	
<code>short,short int,signed short,signed short int</code>	<code>short int</code>		-	
<code>unsigned short,unsigned short int</code>	<code>unsigned short</code>		-	
	<code>int</code>			
<code>int,signed,signed int</code>	<code>int</code>			（常见于 Win16 API）
<code>unsigned,unsigned int</code>	<code>unsigned int</code>			（常见于 Win16 API）
<code>long,long int,signed long,signed long int</code>	<code>long int</code>			（常见于 64 位 Linux、macOS）
<code>unsigned long,unsigned long int</code>	<code>unsigned long</code>			（常见于 64 位 Linux、macOS）
	<code>int</code>			
<code>long long,long long int,signed long long,signed long long int</code>	<code>long long int</code>		-	
<code>unsigned long long,unsigned long long int</code>	<code>unsigned long long int</code>		-	
	<code>unsigned long</code>			
	<code>long int</code>			

当位宽为 时，有符号类型的表示范围为，无符号类型的表示范围为。具体而言，有下表：

位宽 表示范围

有符号：，无符号：

有符号：，无符号：

位宽 表示范围

有符号：, 无符号：

有符号：, 无符号：

▼ 等价类型表述

在不引发歧义的情况下，允许省略部分修饰关键字，或调整修饰关键字的顺序。这意味着同一类型会存在多种等价表述。

例如 `int`, `signed`, `int signed`, `signed int` 表示同一类型，而 `unsigned long` 和 `unsigned long int` 表示同一类型。

另外，一些编译器实现了扩展整数类型，如 GCC 实现了 128 位整数：有符号版的 `__int128_t` 和无符号版的 `__uint128_t`，如果您在比赛时想使用这些类型，**请仔细阅读比赛规则** 以确定是否允许或支持使用扩展整数类型。

▼ 注意

STL 不一定对扩展整数类型有足够的支持，故使用扩展整数类型时需格外小心。

▼ 示例代码

```
1 #include <LTcmath>
2 #include <LTiostream>
3
4 int f1(int n) {
5     return abs(n); // Good
6 }
7
8 int f2(int n) {
9     return std::abs(n); // Good
10 }
11
12 __int128_t f3(__int128_t n) {
13     return abs(n); // Bad
14 }
15
16 // Wrong
17 // __int128_t f4(__int128_t n) {
18 //     return std::abs(n);
19 // }
20
21 int main() {
22     std::cout << "f1: " << f1(-42) << std::endl;
23     std::cout << "f2: " << f2(-42) << std::endl;
24     // std::cout << "f3: " << f3(-42) << std::endl; // Wrong
25     // std::cout << "f4: " << f4(-42) << std::endl; // Wrong
26     return 0;
27 }
```

以上示例代码存在如下问题：

1. `__int128_t f3(__int128_t)` 中使用的是 C 风格的绝对值函数，其签名为 `int abs(int)`，故 `n` 首先会强制转换为 `int`，然后才会调用 `abs` 函数。
2. `__int128_t f4(__int128_t)` 中使用的是 C++ 风格的绝对值函数，其并没有签名为 `__int128_t std::abs(__int128_t)` 的函数重载，所以无法通过编译。
3. C++ 的流式输出不支持 `__int128_t` 与 `__uint128_t`。

以下是一种解决方案：

► 修正后的代码

字符类型

分为「窄字符类型」和「宽字符类型」，由于算法竞赛几乎不会用到宽字符类型，故此处仅介绍窄字符类型。

窄字符型位数一般为 8 位，实际上底层存储方式仍然是整数，一般通过 [ASCII 编码](#) 实现字符与整数的一一对应，有如下三种：

- `signed char`：有符号字符表示的类型，表示范围在 `-128` 到 `127` 之间。
- `unsigned char`：无符号字符表示的类型，表示范围在 `0` 到 `255` 之间。
- `char` 拥有与 `signed char` 或 `unsigned char` 之一相同的表示和对齐，但始终是独立的类型。

`char` 的符号性取决于编译器和目标平台：ARM 和 PowerPC 的默认设置通常没有符号，而 x86 与 x64 的默认设置通常有符号。

GCC 可以在编译参数中添加 `-fsigned-char` 或 `-funsigned-char` 指定将 `char` 视作 `signed char` 或 `unsigned char`，其他编译器请参照文档。需要注意指定与架构默认值不同的符号有可能会破坏 ABI，造成程序无法正常工作。

▼ 注意

与其他整型不同，`char`、`signed char`、`unsigned char` 是三种不同的类型。

一般来说 `signed char`、`unsigned char` 不应用来存储字符，绝大多数情况下，这两种类型均被视作整数类型。

浮点类型

用于存储「实数」（注意并不是严格意义上的实数，而是实数在一定规则下的近似），包括以下三种：

- `float`：单精度浮点类型。如果支持就会匹配 IEEE-754 binary32 格式。
- `double`：双精度浮点类型。如果支持就会匹配 IEEE-754 binary64 格式。
- `long double`：扩展精度浮点类型。如果支持就会匹配 IEEE-754 binary128 格式，否则如果支持就会匹配 IEEE-754 binary64 扩展格式，否则匹配某种精度优于 binary64 而值域至少和 binary64 一样好的非 IEEE-754 扩展浮点格式，否则匹配 IEEE-754 binary64 格式。

浮点格式	位宽	最大正数	精度位数
IEEE-754 binary32 格式	32	3.4e+38	24
IEEE-754 binary64 格式	64	1.7e+308	53
IEEE-754 binary64 扩展格式	80	1.7e+308	64
IEEE-754 binary128 格式	128	1.7e+308	113

IEEE-754 浮点格式的最小负数是最大正数的相反数。

因为 `float` 类型表示范围较小，且精度不高，实际应用中常使用 `double` 类型表示浮点数。

另外，浮点类型可以支持一些特殊值：

- 无穷（正或负）：`INFINITY`。
- 负零：`-0.0`，例如 `1.0 / 0.0 == INFINITY`，`1.0 / -0.0 == -INFINITY`。
- 非数（NaN）：`std::nan`，一般可以由 `0.0 / 0.0` 之类的运算产生。它与任何值（包括自身）比较都不相等，C++11 后可以使用 `std::isnan` 判断一个浮点数是不是 NaN。

无类型

`void` 类型为无类型，与上面几种类型不同的是，不能将一个变量声明为 `void` 类型。但是函数的返回值允许为 `void` 类型，表示该函数无返回值。

空指针类型

请参阅指针的 [对应章节](#)

定宽整数类型

C++11 起提供了定宽整数的支持，具体如下：

- `<LTcstdint>`：提供了若干定宽整数的类型和各定宽整数类型最大值、最小值等的宏常量。
- `<LTcinttypes>`：为定宽整数类型提供了用于 `std::fprintf` 系列函数和 `std::fscanf` 系列函数的格式宏常量。

定宽整数有如下几种：

- `intN_t`：宽度 **恰为** `N` 位的有符号整数类型，如 `int32_t`。
- `int_fastN_t`：宽度 **至少** 有 `N` 位的 **最快的** 有符号整数类型，如 `int_fast32_t`。
- `int_leastN_t`：宽度 **至少** 有 `N` 位的 **最小的** 有符号整数类型，如 `int_least32_t`。

无符号版本只需在有符号版本前加一个字母 `u` 即可，如 `uint32_t`, `uint_least8_t`。

标准规定必须实现如下 16 种类型：

```
int_fast8_t,int_fast16_t,int_fast32_t,int_fast64_t,

int_least8_t,int_least16_t,int_least32_t,int_least64_t,

uint_fast8_t,uint_fast16_t,uint_fast32_t,uint_fast64_t,

uint_least8_t,uint_least16_t,uint_least32_t,uint_least64_t.
```

绝大多数编译器在此基础上都实现了如下 8 种类型：

```
int8_t,int16_t,int32_t,int64_t,

uint8_t,uint16_t,uint32_t,uint64_t.
```

在实现了对应类型的情况下，C++ 标准规定必须实现表示对应类型的最大值、最小值、位宽的宏常量，格式为将类型名末尾的 `_t` 去掉后转大写并添加后缀：

- `_MAX` 表示最大值，如 `INT32_MAX` 即为 `int32_t` 的最大值。
- `_MIN` 表示最小值，如 `INT32_MIN` 即为 `int32_t` 的最小值。

▼ 注意

定宽整数类型本质上是普通整数类型的类型别名，所以混用定宽整数类型和普通整数类型可能会影响跨平台编译，例如：

▼ 示例代码

```
1 #include <LTAgorithm>
2 #include <LTcstdint>
3 #include <LTiostream>
4
5 int main() {
6     long long a;
7     int64_t b;
8     std::cin >> a >> b;
9     std::cout << std::max(a, b) << std::endl;
10    return 0;
11 }
```

`int64_t` 在 64 位 Windows 下一般为 `long long int`，而在 64 位 Linux 下一般为 `long int`，所以这段代码在使用 64 位 Linux 下的 GCC 时不能通过编译，而使用 64 位 Windows 下的 MSVC 时可以通过编译，因为 `std::max` 要求输入的两

个参数类型必须相同。

此外，C++17 起在 `<limits>` 中提供了 `std::numeric_limits` 类模板，用于查询各种算数类型的属性，如最大值、最小值、是否是整形、是否有符号等。

```
1 #include <cstdint>
2 #include <limits>
3
4 std::numeric_limits<int32_t>::max(); // int32_t 的最大值, 2'147'483'647
5 std::numeric_limits<int32_t>::min(); // int32_t 的最小值, -2'147'483'648
6
7 std::numeric_limits<double>::min(); // double 的最小值, 约为 2.22507e-308
8 std::numeric_limits<double>::epsilon(); // 1.0 与 double 的下个可表示值的差,
9 // 约为 2.22045e-16
```

类型转换

在一些时候（比如某个函数接受 `int` 类型的参数，但传入了 `double` 类型的变量），我们需要将某种类型，转换成另外一种类型。

C++ 中类型的转换机制较为复杂，这里主要介绍对于基础数据类型的两种转换：数值提升和数值转换。

数值提升

数值提升过程中，值本身保持不变。

▼ Note

C 风格的可变参数域在传值过程中会进行默认参数提升。如：

▼ 示例代码

```

1 #include <LTstdarg.h>
2 #include <LTstdio.h>
3
4 void test(int tot, ...) {
5     va_list valist;
6     int i;
7
8     // 初始化可变参数列表
9     va_start(valist, tot);
10
11     for (i = 0; i < tot; ++i) {
12         // 获取第 i 个变量的值
13         double xx = va_arg(valist, double); // Correct
14         // float xx = va_arg(valist, float); // Wrong
15
16         // 输出第 i 个变量的底层存储内容
17         printf("i = %d, value = 0x%016llx\n", i, *(long long *)(&xx));
18     }
19
20     // 清理可变参数列表的内存
21     va_end(valist);
22 }
23
24 int main() {
25     float f;
26     double fd, d;
27     f = 123.; // 0x42f60000
28     fd = 123.; // 0x405ec00000000000
29     d = 456.; // 0x407c800000000000
30     test(3, f, fd, d);
31 }

```

在调用 `test` 时，`f` 提升为 `double`，从而底层存储内容和 `fd` 相同，输出为

```

1 i = 0, value = 0x405ec00000000000
2 i = 1, value = 0x405ec00000000000
3 i = 2, value = 0x407c800000000000

```

若将 `double xx = va_arg(valist, double);` 改为 `float xx = va_arg(valist, float);`，GCC 应该给出一条类似下文的警告：

```

1 In file included from test.c:2:
2 test.c: In function 'test':
3 test.c:14:35: warning: 'float' is promoted to 'double' when passed through '...'
4   14 |         float xx = va_arg(valist, float);
5      |                                ^
6 test.c:14:35: note: (so you should pass 'double' not 'float' to 'va_arg')
7 test.c:14:35: note: if this code is reached, the program will abort

```

此时的程序将会在输出前终止。

这一点也能解释为什么 `printf` 的 `%f` 既能匹配 `float` 也能匹配 `double`。

整数提升

小整数类型（如 `char`）的纯右值可转换成较大整数类型（如 `int`）的纯右值。

具体而言，算术运算符不接受小于 `int` 的类型作为它的实参，而在左值到右值转换后，如果适用就会自动实施整数提升。

具体地，有如下规则：

- 源类型为 `signed char`、`signed short` / `short` 时，可提升为 `int`。
- 源类型为 `unsigned char`、`unsigned short` 时，若 `int` 能保有源类型的值范围，则可提升为 `int`，否则可提升为 `unsigned int`。（C++20 起 `char8_t` 也适用本规则）
- `char` 的提升规则取决于其底层类型是 `signed char` 还是 `unsigned char`。
- `bool` 类型可转换到 `int`：`false` 变为 0，`true` 变为 1。
- 若目标类型的值范围包含源类型，且源类型的值范围不能被 `int` 和 `unsigned int` 包含，则源类型可提升为目标类型。³

▼ 注意

`char`->`short` 不是数值提升，因为 `char` 要优先提升为 `int` / `unsigned int`，之后是 `int` / `unsigned int`->`short`，不满足数值提升的条件。

如（以下假定 `int` 为 32 位，`unsigned short` 为 16 位，`signed char` 和 `unsigned char` 为 8 位，`bool` 为 1 位）

- `(signed char)'\0' - (signed char)'\xff'` 会先将 `(signed char)'\0'` 提升为 `(int)0`、将 `(signed char)'\xff'` 提升为 `(int)-1`，再进行 `int` 间的运算，最终结果为 `(int)1`。
- `(unsigned char)'\0' - (unsigned char)'\xff'` 会先将 `(unsigned char)'\0'` 提升为 `(int)0`、将 `(unsigned char)'\xff'` 提升为 `(int)255`，再进行 `int` 间的运算，最终结果为 `(int)-255`。
- `false - (unsigned short)12` 会先将 `false` 提升为 `(int)0`、将 `(unsigned short)12` 提升为 `(int)12`，再进行 `int` 间的运算，最终结果为 `(int)-12`。

浮点提升

位宽较小的浮点数可以提升为位宽较大的浮点数（例如 `float` 类型的变量和 `double` 类型的变量进行算术运算时，会将 `float` 类型变量提升为 `double` 类型变量），其值不变。

数值转换

数值转换过程中，值可能会发生改变。

▼ 注意

数值提升优先于数值转换。如 `bool`->`int` 时是数值提升而非数值转换。

整数转换

- 如果目标类型为位宽为 的无符号整数类型，则转换结果是原值 后的结果。
 - 若目标类型位宽大于源类型位宽：
 - 若源类型为有符号类型，一般情况下需先进行符号位扩展再转换。

如

- 将 `(short)-1` (`(short)0b1111'1111'1111'1111`) 转换为 `unsigned int` 类型时，先进行符号位扩展，得到 `0b1111'1111'1111'1111'1111'1111'1111'1111`，再进行整数转换，结果为 `(unsigned int)4'294'967'295` (`(unsigned int)0b1111'1111'1111'1111'1111'1111'1111'1111`)。
- 将 `(short)32'767` (`(short)0b0111'1111'1111'1111`) 转换为 `unsigned int` 类型时，先进行符号位扩展，得到 `0b0000'0000'0000'0000'0111'1111'1111'1111`，再进行整数转换，结果为 `(unsigned int)32'767` (`(unsigned int)0b0000'0000'0000'0000'0111'1111'1111'1111`)。
- 若源类型为无符号类型，则需先进行零扩展再转换。

如将 `(unsigned short)65'535` (`(unsigned short)0b1111'1111'1111'1111`) 转换为 `unsigned int` 类型

时，先进行零扩展，得到 `0b0000'0000'0000'0000'1111'1111'1111'1111`，再进行整数转换，结果为 `(unsigned int)65'535` (`(unsigned int)0b0000'0000'0000'0000'1111'1111'1111'1111`)。

- 若目标类型位宽不大于源类型位宽，则需先截断再转换。

如将 `(unsigned int)4'294'967'295` (`(unsigned int)0b1111'1111'1111'1111'1111'1111'1111'1111`) 转换为 `unsigned short` 类型时，先进行截断，得到 `0b1111'1111'1111'1111`，再进行整数转换，结果为 `(unsigned short)65'535` (`(unsigned short)0b1111'1111'1111'1111`)。

- 如果目标类型为位宽为 的带符号整数类型，则 **一般情况下**，转换结果可以认为是原值 后的结果。⁴

例如将 `(unsigned int)4'294'967'295` (`(unsigned int)0b1111'1111'1111'1111'1111'1111'1111'1111`) 转换为 `short` 类型时，结果为 `(short)-1` (`(short)0b1111'1111'1111'1111`)。

- 如果目标类型是 `bool`，则是 [布尔转换](#)。
- 如果源类型是 `bool`，则 `false` 转为对应类型的 0，`true` 转为对应类型的 1。

浮点转换

位宽较大的浮点数转换为位宽较小的浮点数，会将该数舍入到目标类型下最接近的值。

浮点整数转换

- 浮点数转换为整数时，会舍弃浮点数的全部小数部分。

如果目标类型是 `bool`，则是 [布尔转换](#)。

- 整数转换为浮点数时，会舍入到目标类型下最接近的值。

如果该值不能适应到目标类型中，那么行为未定义。

如果源类型是 `bool`，那么 `false` 转换为零，而 `true` 转换为一。

布尔转换

将其他类型转换为 `bool` 类型时，零值转换为 `false`，非零值转换为 `true`。

定义变量

简单地说⁵，定义一个变量，需要包含类型说明符（指明变量的类型），以及要定义的变量名。

例如，下面这几条语句都是变量定义语句。

```
1 int oi;
2 double wiki;
3 char org = 'c';
```

在目前我们所接触到的程序段中，定义在花括号包裹的地方的变量是局部变量，而定义在没有花括号包裹的地方的变量是全局变量。实际有例外，但是现在不必了解。

定义时没有初始化值的全局变量会被初始化为 `0`。而局部变量没有这种特性，需要手动赋初始值，否则可能引起难以发现的 bug。

变量作用域

作用域是变量可以发挥作用的代码块。

全局变量的作用域，自其定义之处开始⁶，至文件结束位置为止。

局部变量的作用域，自其定义之处开始，至代码块结束位置为止。

由一对大括号括起来的若干语句构成一个代码块。

```
1 int g = 20; // 定义全局变量
2
3 int main() {
4     int g = 10; // 定义局部变量
5     printf("%d\n", g); // 输出 g
6     return 0;
7 }
```

如果一个代码块的内嵌块中定义了相同变量名的变量，则内层块中将无法访问外层块中相同变量名的变量。

例如上面的代码中，输出的 的值将是 。因此为了防止出现意料之外的错误，请尽量避免局部变量与全局变量重名的情况。

常量

常量是固定值，在程序执行期间不会改变。

常量的值在定义后不能被修改。定义时加一个 `const` 关键字即可。

```
1 const int a = 2;
2 a = 3;
```

如果修改了常量的值，在编译环节就会报错：`error: assignment of read-only variable 'a'`。

参考资料与注释

1. [Working Draft, Standard for Programming Language C++](#)
2. [类型 - cppreference.com](#)
3. C 语言的 [算术类型 - cppreference.com](#)
4. [基础类型 - cppreference.com](#)
5. [定宽整数类型（C++11 起） - cppreference.com](#)
6. William Kahan (1 October 1997). "[Lecture Notes on the Status of IEEE Standard 754 for Binary Floating-Point Arithmetic](#)".
7. [隐式转换 - cppreference.com](#)
8. [声明 - cppreference](#)
9. [作用域 - cppreference.com](#)

-
1. 参见 <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3054.pdf> ↩
 2. 包括数组类型、引用类型、指针类型、类类型、函数类型等。由于本篇文章是面向初学者的，故不在本文做具体介绍。具体请参阅 [类型 - cppreference.com](#) ↩
 3. 不包含宽字符类型、位域和枚举类型，详见 [整型转换 - cppreference](#)。↩
 4. 自 C++20 起生效。C++20 前结果是实现定义的。详见 [整型转换 - cppreference](#)。↩
 5. 定义一个变量时，除了类型说明符之外，还可以包含其他说明符。详见 [声明 - cppreference](#)。↩
 6. 更准确的说法是 [声明点](#)。↩
-

本页面最近更新：2024/4/19 14:58:08, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [abc1763613206](#), [CamberLoid](#), [CCXXXI](#), [CoelacanthusHex](#), [Enter-tainer](#), [Friendseeker](#), [Great-designer](#), [Haohu Shen](#), [lr1d](#), [ksyx](#), [mgt](#), [orzAtalod](#), [shuzhouliu](#), [StudyingFather](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用